# StructPaper v2 — Đề xuất kiến trúc & design pattern

*Căn cứ: Yêu_cầu_phần_mềm.docx, Template_EN.docx, Template_VI.docx, DOCS_HIEN_TRANG_UPLOAD_EDITOR.md, DOCS_KIEN_TRUC_LUONG_DU_LIEU_UPLOAD_EDITOR.md và PDF mẫu ID630_HS.*

## 0. Tóm tắt điều hành

Hiện trạng đã chạy được một đường ống trọn vẹn **Upload → CompletePaper → Editor → LaTeX → PDF**. Nhưng ba thứ đang dính chặt vào nhau nên nhiều người không thể làm song song:

1. Logic đọc DOCX nằm trong một file hơn 2.100 dòng ( `docx-parser.ts` ).
2. Mô hình dữ liệu quá phẳng (chuỗi text, ảnh base64, số thứ tự lưu cứng).
3. `UnifiedEditorView` điều phối mọi thứ (state, lưu, biên dịch, dàn trang).

Mọi yêu cầu lớn trong file yêu cầu (cross-reference, đánh số lại equation, renumber citation, JATS XML, nhiều journal, revision, preflight, batch) đều cần **một mô hình tài liệu có cấu trúc và ID ổn định**. Vì vậy đó là thứ phải làm đầu tiên.

**Bảy quyết định lớn được đề xuất:**

1. Tách **lõi miền** ( `doc-model` ) thuần TypeScript, không dính React, DOM, IndexedDB hay HTTP.
2. **Manuscript v2**: cây block + inline node + ID ổn định; số Figure/Table/Equation/Reference là dữ liệu *dẫn xuất*, không lưu cứng.
3. **Ingest DOCX là pipeline nhiều tầng**: ưu tiên đọc theo *style Word của template*, sau đó heuristic, cuối cùng mới AI. Chạy trong Web Worker và viết dạng isomorphic để dùng lại cho batch phía server.
4. **Journal Profile là dữ liệu** (JSON có schema), không phải code. Thêm tạp chí mới = thêm profile và một file `.cls` .
5. **Editor = Store + Command + Feature module + Block registry** (undo/redo, autosave, op-log đồng bộ BE có sẵn).
6. **Persistence offline-first**: IndexedDB (Dexie) đứng sau interface `Repository` ; sau này Spring Boot/MySQL chỉ là thêm một adapter.
7. **Compile service tách riêng**, sandbox, có cache/queue/cancel; dùng chung cho PDF preview và PDF xuất bản (PDF/X).

# 1. Phát hiện từ tài liệu (các bất cập cần xử lý)

| # | Phát hiện | Bằng chứng trong tài liệu | Hệ quả | Hướng xử lý |
|---|---|---|---|---|
| 1 | Parser một cục | `docx-parser.ts` >2.100 dòng gồm mammoth, JSZip, OMML, metadata, bảng, ảnh, reference, nhận diện journal, merge AI | Không chia việc được; sửa bảng có thể vỡ ảnh; khó test | Pipeline nhiều tầng (mục 4) |
| 2 | Parse XML bằng regex | Mô tả dùng regex + DOM; `ommlToLatex` chỉ có 6 cấu trúc (f, sSup, sSub, sSubSup, rad, d) | Vỡ với matrix, tổng/tích phân, accent, MathType, equation array (yêu cầu mục 4 nêu rõ) | Parser XML thật; OMML → MathML → LaTeX; giữ OMML gốc |
| 3 | **Bỏ phí style của template** | Template_EN/VI có ~35 style đặt tên: ArticleTitle, Authors, Affiliation, CorrespondingAuthor, Date*, Abstract, Keywords, Heading1-3, Equation, TableCaption, TableNote, SourceLine, FigureCaption, Section*, Reference. Các style `Guide*`, `Instruction` phải bị bỏ qua | Tác giả đã dùng đúng template mà parser vẫn đoán bằng heuristic/AI; sai và tốn token | Recognizer theo style, độ tin cậy 1.0 |
| 4 | Model phẳng | `content: string` chứa `[1, 2]` thô; không có inline format; `PaperSection` không có equation/list/footnote; hình/bảng không có `source`; không có block cho Ethics/Funding/Contributions/Conflict/Data availability (template có đủ) | Mất *in nghiêng* (H. pylori trong PDF mẫu), chỉ số dưới ($FEV_1$); không cross-ref; renumber bằng regex | Manuscript v2 (mục 3) |
| 5 | Hai biểu diễn bảng | `headers/rows` cùng `headerRows`; `normalizePaperTable` đoán header 2 tầng từ chuỗi kiểu `A / B` | Dữ liệu suy đoán, dễ sai, khó làm longtable, bảng ngang, repeat header | Một biểu diễn lưới (cell có rowSpan/colSpan) |
| 6 | Ảnh là base64 trong JSON | `src: data:...`; ép mime png/jpeg; lưu `sessionStorage` (~5 MB); `catch {}` nuốt lỗi; gửi lại base64 mỗi lần compile; alias `fig1.png/fig1.jpg` | Bài nhiều ảnh mất dữ liệu **im lặng** khi lưu; EMF/WMF/TIFF/EPS/PDF (yêu cầu) bị gán nhầm mime; không kiểm tra DPI | Asset store theo hash nội dung |
| 7 | `.doc` | UI nhận `.doc` nhưng JSZip/mammoth không đọc được `.doc` nhị phân (OLE2) | Lỗi khi người dùng chọn `.doc` | Chuyển đổi phía server (LibreOffice) hoặc từ chối rõ ràng |
| 8 | State 2 tầng + sessionStorage | `formData` ↔ `PaperContext`; `isDirty` set tay; lưu bằng Ctrl+S | Không undo/redo, không autosave, không lịch sử revision, đóng tab là mất | Store + Command + IndexedDB |
| 9 | Hai bộ sinh LaTeX | `exporters.ts` (`exportToLatex`) và `latex-template.ts` (`generateFullJournalLatex`) | Trùng lặp, dễ lệch | Một LaTeX emitter duy nhất |
| 10 | Compile ở hai nơi, chưa sandbox | Next route và Spring cùng compile; thay font bằng string replace; chạy `xelatex` trên TeX do client gửi | Rủi ro `\write18` / `\input` đọc file hệ thống; không queue/cancel/cache | Compiler service (mục 8) |
| 11 | Lộ API key, lộ bản thảo | `NEXT_PUBLIC_OPENAI_API_KEY`, người dùng gõ `sk-…` | Khóa nằm ở client; bản thảo chưa công bố đi thẳng ra ngoài | AI qua proxy BE, có consent |

| # | Phát hiện | Bằng chứng trong tài liệu | Hệ quả | Hướng xử lý |
|---|---|---|---|---|
| 12 | Quy tắc journal rải rác và mâu thuẫn | Tóm tắt 150–250 từ (app) vs 275–500 từ (template); từ khóa dấu phẩy vs chấm phẩy; ngày `02-10-2026` vs `dd/mm/yyyy`; HS có 8 loại bài vs template 4 loại; trích dẫn `[1]` (template) vs chỉ số trên (PDF mẫu) | Mỗi nơi một con số | Journal Profile là nguồn sự thật duy nhất |
| 13 | Song ngữ cứng | Model có `title/titleEn`; template lại tách thành 2 file EN và VI | Không mở rộng được | `lang` chính + `translations` |
| 14 | PDF mẫu có lỗi dữ liệu | DOI ghi `…v7i1.630` nhưng bài là `7(2)` | Chính là loại lỗi preflight phải bắt | Rule QC đối chiếu DOI với volume/issue; dùng PDF mẫu làm golden test thị giác |

---

# 2. Kiến trúc tổng thể

## 2.1 Phân tầng (Hexagonal / Ports & Adapters)

```
apps/web  (Next.js, vỏ mỏng: route, layout, DI)
   |
packages/feature-*   UI theo tính năng (metadata, authors, body, references,
qc, layout, preview, export)
   |
packages/editor-core store, command bus, history, block registry, autosave
   |
packages/ingest-docx · exporters · qc-engine · journal-profiles · ai-client ·
storage · compiler-client
   |
packages/doc-model   types, zod schema, numbering, visitors, migrations   ←
không phụ thuộc gì

apps/api (Spring Boot 3, modular monolith)      services/compiler-worker (TeX
Live/Tectonic, sandbox)
```

**Luật phụ thuộc:** mũi tên chỉ đi xuống. `doc-model` không import gì; `ingest`, `exporters`, `qc` chỉ phụ thuộc `doc-model`. Ép luật bằng `dependency-cruiser` hoặc `eslint-plugin-boundaries` trong CI; vi phạm thì fail build. Đây là điều kiện để nhiều người làm song song không đạp lên nhau.

## 2.2 Cổng (port) và bộ chuyển đổi (adapter)

| Port (interface) | Adapter giai đoạn IndexedDB | Adapter giai đoạn Spring Boot |
|---|---|---|
| `ManuscriptRepository` | `IndexedDbManuscriptRepo` (Dexie) | `HttpManuscriptRepo` + decorator `SyncedRepo` |
| `AssetRepository` | Blob trong IndexedDB | Upload qua API → S3/MinIO hoặc filesystem |
| `CompilerPort` | `LocalCliCompiler` (chỉ dev) | `RemoteCompiler` (job qua API) |
| `AiPort` | `MockAi` / bật tắt | Proxy BE → OpenAI |
| `JournalProfileProvider` | JSON đóng gói sẵn | API `/journals` có version |
| `DocConverterPort` (.doc → .docx) | không hỗ trợ, báo lỗi rõ | LibreOffice headless phía server |
| `Clock`, `IdGenerator` | thật | giả (để test ổn định) |

UI và core chỉ biết *interface*. Đổi nơi lưu hoặc nơi biên dịch không phải sửa Editor.

## 2.3 Cấu trúc monorepo (pnpm workspaces + Turborepo)

```
structpaper/
├─ apps/
│  ├─ web/                  # Next.js: chỉ routing + lắp ráp (composition root)
│  └─ api/                  # Spring Boot
├─ services/
│  └─ compiler-worker/      # Docker: texlive-xetex + fonts, chạy job biên dịch
├─ packages/
│  ├─ doc-model/            # Manuscript v2, zod, numbering, visitor, migrations
│  ├─ ingest-docx/          # pipeline đọc Word (mục 4)
│  ├─ journal-profiles/     # JSON + schema + loader (mục 5)
│  ├─ exporters/            # latex, docx, jats, bibtex, zip, html
│  ├─ qc-engine/            # rule engine + rules (mục 7)
│  ├─ editor-core/          # store, commands, history, block registry
│  ├─ storage/              # repository interfaces + dexie + http + sync
```

```
|  ├─ compiler-client/        # CompilerPort + client cho API
|  ├─ ai-client/              # AiPort + prompt có version
|  ├─ feature-metadata/ feature-authors/ feature-abstract/
|  ├─ feature-body/           # heading, paragraph, figure, table, equation
|  ├─ feature-references/ feature-qc/ feature-layout/ feature-preview/
feature-export/
|  └─ ui-kit/                # Select, DatePicker, Modal, Dock...
├─ fixtures/                  # docx mẫu + JSON kỳ vọng + PDF tham chiếu
(golden)
├─ contracts/                 # openapi.yaml + JSON Schema sinh từ zod
└─ CODEOWNERS                 # chia quyền sở hữu theo package
```

---

# 3. Mô hình dữ liệu Manuscript v2 ( `doc-model` )

## 3.1 Nguyên tắc

- **ID ổn định cho mọi block/inline đích tham chiếu** (ULID). Cross-reference và citation trỏ bằng ID, không bằng số.
- **Số thứ tự là dữ liệu dẫn xuất.** Figure 1, Table 2, Equation (3), `[1]` … `[n]` được tính bằng hàm `computeNumbering(doc)` từ vị trí và thứ tự xuất hiện lần đầu. Xóa/chèn giữa bài thì mọi thứ tự cập nhật theo; production editor không sửa tay (đúng yêu cầu mục 4 và 5).
- **Nội dung văn bản là cây inline**, không phải chuỗi: giữ được đậm/nghiêng/chỉ số, citation, cross-ref, công thức nội dòng.
- **Ảnh không nằm trong JSON**; chỉ có `assetId = sha256`.
- **Có `schemaVersion` + migration** (vì dữ liệu nằm lâu dài trong IndexedDB người dùng và MySQL).
- Dữ liệu nguồn gốc (provenance) được giữ lại: block nào lấy từ đâu, độ tin cậy bao nhiêu, để UI tô vàng chỗ cần người kiểm.

## 3.2 Hình dạng (rút gọn)

```
type Id = string; // ULID

interface Manuscript {
  schemaVersion: 2;
  id: Id;
  lang: 'vi' | 'en';                  // ngôn ngữ chính của bài (khớp
```

```
Template_VI/EN)
  journalId: string;
  articleType: string;                // lấy từ JournalProfile.articleTypes
  meta: FrontMatter;
  body: Section[];                    // cây section (tương ứng <sec> của
JATS)
  back: Statement[];                  // ethics, acknowledgments, funding,
contributions, conflict, data
  references: Reference[];            // cấu trúc hóa, không chỉ chuỗi thô
  assets: Record<string, AssetMeta>;  // sha256 -> {mime, w, h, dpi, vector}
  layoutOverrides?: LayoutOverrides;  // chỉ phần khác profile
  provenance?: Record<Id, Provenance>;
}

interface FrontMatter {
  title: Inline[]; translations?: { en?: Inline[] };
  authors: Author[];                  // {id, name, affiliationIds,
corresponding, email, orcid}
  affiliations: Affiliation[];        // {id, text, translations}
  abstract: Inline[][];               // 1 đoạn hoặc có cấu trúc
  keywords: string[];
  dates: { received?; revised?; accepted?; published? }; // ISO 8601 bên
trong, định dạng theo profile khi hiển thị
  doi?: string; volume?: string; issue?: string; pages?: [number, number];
}

type Block =
  | { type: 'paragraph'; id: Id; content: Inline[] }
  | { type: 'list'; id: Id; ordered: boolean; items: Block[][] }
  | { type: 'quote' | 'box'; id: Id; content: Block[] }
  | { type: 'equation'; id: Id; latex: string; omml?: string; numbered:
boolean; label?: string; status: 'ok' | 'fallback-image' }
  | { type: 'figure'; id: Id; assetId: string; caption: Inline[]; source?:
Inline[]; panels?: Panel[]; layout: FigureLayout }
  | { type: 'table'; id: Id; caption: Inline[]; grid: Cell[][]; headerRows:
number; notes?: Inline[]; source?: Inline[]; layout: TableLayout }
  | { type: 'pageBreak'; id: Id };

interface Section { id: Id; level: 1 | 2 | 3; title: Inline[]; blocks:
Block[]; children: Section[] }

type Inline =
  | { t: 'text'; v: string; marks?: ('b' | 'i' | 'sup' | 'sub' | 'code')[] }
```

```
  | { t: 'cite'; refIds: Id[] }                // hiển thị [1], [2-4] hoặc
chỉ số trên theo profile
  | { t: 'xref'; target: Id; kind: 'figure' | 'table' | 'equation' |
'section' }
  | { t: 'math'; latex: string }
  | { t: 'footnote'; blocks: Block[] }
  | { t: 'link'; href: string; content: Inline[] };

interface Cell { content: Inline[]; rowSpan: number; colSpan: number; align?:
'l' | 'c' | 'r'; header?: boolean }
interface Reference { id: Id; raw: string; csl?: CslJson; doi?: string; url?:
string; parse: 'ok' | 'partial' | 'failed' }
```

### 3.3 Dữ liệu dẫn xuất (không lưu)

```
computeNumbering(doc, citationStyle) → {
  figure: Map<Id, number>, table: Map<Id, number>, equation: Map<Id, number>,
  citationOrder: Id[],                // thứ tự xuất hiện lần đầu (kiểu đánh
số dùng làm [n], QC luôn dùng)
  citationLabels: Map<Id, string>,    // nhãn do CitationStyle tính: '3',
'2-4' hoặc 'Nguyễn, 2020'
  bibliographyOrder: Id[],            // theo thứ tự cite (Vancouver) hoặc
alphabet (APA)
  uncitedRefs: Id[], danglingCites: Id[], xrefBroken: Id[],
  firstMentionBeforeFloat: Map<Id, boolean>  // template yêu cầu nhắc trước
khi hình/bảng xuất hiện
}
```

Hàm này là *pure function*, memoize theo phiên bản document, dùng chung cho editor, QC và mọi exporter. Đây là lời giải cho ví dụ xóa Reference 5 hoặc chèn Equation giữa bài.

### 3.4 Di chuyển từ `CompletePaper` v1

Viết `migrateV1toV2(CompletePaper): Manuscript` (tách `[1,2]` trong chuỗi thành node `cite`, tách base64 thành asset, chuyển `headerRows` thành lưới). Giữ adapter ngược tạm thời để UI cũ vẫn chạy trong lúc chuyển (xem lộ trình, mục 14).

## 3.5 Hệ thống trích dẫn mở rộng được (Vancouver, APA, IEEE…)

Kiểu trích dẫn là cấu hình, không phải dữ liệu của bài. Bài chỉ lưu node `cite` trỏ tới `refIds`; cách hiển thị do một `CitationStyle` quyết định. Nhờ vậy chuyển từ Vancouver sang APA không phải sửa `Manuscript`, Editor, QC hay exporter, chỉ thêm một style.

Vancouver là kiểu đánh số theo thứ tự xuất hiện lần đầu. APA là kiểu tác giả–năm, danh mục xếp theo alphabet. Hai kiểu khác nhau ở bốn chỗ: nhãn trong bài, thứ tự danh mục, dữ liệu tối thiểu của reference và cách nhận diện khi đọc Word. Kiến trúc xử lý cả bốn chỗ như sau.

```
interface CitationStyle {
  id: string;                                    // 'vancouver' | 'apa7' |
'ieee' ...
  kind: 'numeric' | 'author-year';
  sortBibliography: 'by-first-cite' | 'alphabetical';
  formatInText(cite: CiteNode, ctx: CitationContext): Inline[];   // [2–4],
chỉ số trên, (Nguyễn và cs., 2020)
  formatBibliography(refs: Reference[], ctx: CitationContext): Block[];
}
```

Cách triển khai khuyến nghị: một `CslCitationStyle` dùng **CSL** (Citation Style Language) với `citeproc-js`. Vancouver, APA, IEEE… đều là file `.csl` có sẵn, nên thêm kiểu mới là thêm một file style và một dòng trong profile, không viết code. Profile chỉ khai báo:

```
citation: { style: 'apa7', locale: 'vi-VN', inText: 'bracket' | 'superscript'
| 'parenthetical' }
```

| Tầng | Thay đổi khi thêm kiểu trích dẫn mới |
|---|---|
| `doc-model` | Node `cite` thêm `locator` (trang), `prefix`, `suffix`, `suppressAuthor` (dạng *Nguyễn (2020) cho thấy…*). `Reference.csl` trở thành dữ liệu chính, `raw` chỉ giữ bản gốc để đối chiếu |
| `computeNumbering` | Nhận `citationStyle`, trả thêm `citationLabels` và `bibliographyOrder`. Kiểu tác giả–năm thì danh mục theo alphabet nên không còn khái niệm renumber, nhưng `citationOrder` vẫn dùng cho QC |
| `journal-profiles` | `citation.style` + `locale` thay cho giá trị Vancouver cố định |

| Tầng | Thay đổi khi thêm kiểu trích dẫn mới |
|---|---|
| `ingest-docx` | Thêm `AuthorYearCitationRecognizer` (ghép *(Nguyễn, 2020)* với mục trong danh mục) và `ReferenceParser` (chuỗi → CSL). Nhận diện `[1]`, `[2-4]` của Vancouver vẫn giữ nguyên |
| `qc-engine` | Luật theo `kind`: kiểu đánh số kiểm số khớp thứ tự cite; kiểu tác giả–năm kiểm mỗi cite khớp đúng một reference (trùng tác giả–năm cần hậu tố a, b). Thêm luật *reference thiếu trường mà style mới yêu cầu* (APA cần tác giả, năm, tựa đề) |
| `exporters` | Nhận chuỗi đã được `citeproc` format sẵn, không để BibTeX/biblatex tự định dạng. PDF, preview HTML, DOCX và JATS cùng ra một kết quả. JATS vẫn dùng `<xref ref-type="bibr">` |
| Editor | Bộ chọn tài liệu tham khảo và danh mục đọc nhãn từ `citationLabels`; thêm ô *trang* và *ẩn tên tác giả* cho APA |
| `storage`, BE | Không đổi. Chỉ cần lưu id style trong profile (có version) |

Hai lưu ý khi triển khai:

- **Dữ liệu reference quyết định khả năng đổi kiểu.** Bài Vancouver chỉ có chuỗi thô vẫn render được kiểu đánh số, nhưng không thể sang APA nếu chưa parse thành CSL. Vì vậy `ReferenceParser` (có thể kèm tra cứu DOI qua Crossref ở BE) nên làm sớm; reference có `parse: 'partial'` hoặc `'failed'` phải được QC báo trước khi đổi kiểu.
- **Ngôn ngữ ảnh hưởng định dạng.** Tên người Việt (họ đứng trước), *et al.* đổi thành *và cs.* và các chữ như *tr.* hay *Biên tập* được lấy từ `locale` của CSL, nên cần bộ locale tiếng Việt trong `citation.locale`.

---

## 4. Pipeline ingest DOCX (`packages/ingest-docx`)

Ý tưởng: **tách đọc (lossless) khỏi hiểu (semantic)**. Mỗi tầng là một thư mục, một nhóm người, kiểm thử độc lập.

```
.docx → [1 Package] → [2 OOXML→RawDoc] → [3 Recognizers] → [4 Assemblers] → [5 Normalizers] → [6 Validate] → Manuscript + IngestReport
```

| Tầng | Việc | Pattern | Ghi chú |
|---|---|---|---|
| 1. Package | Mở ZIP, đọc các part: `document.xml`, `styles.xml`, `numbering.xml`, `footnotes.xml`, `endnotes.xml`, `comments.xml`, `_rels`, `media/*` | Adapter | Chỉ phụ thuộc thư viện ZIP; trả về `DocxPackage` |
| 2. OOXML → RawDoc | Duyệt XML **một lần** bằng parser thật (isomorphic, ví dụ `fast-xml-parser`, **không dùng DOMParser** để chạy được cả Node). Giải `basedOn` của style. Gỡ `w:sdt`, `mc:AlternateContent`. Xử lý field (`fldChar/instrText`: REF, SEQ, HYPERLINK, `ADDIN EN.CITE`, `ADDIN CSL_CITATION` của Zotero/Mendeley) | Visitor | Kết quả là `RawBlock[]` (đoạn + run + style + numbering + bảng lưới + drawing ref + math) và **chưa có ngữ nghĩa** |
| 2b. Track Changes | Chính sách `acceptAll` (mặc định) / `rejectAll` / `keepAsComments` cho `w:ins`, `w:del`, `w:moveFrom/To` | Strategy | Đáp ứng yêu cầu Input: Word có Track Changes |
| 3. Recognizers | Mỗi recognizer nhìn một `RawBlock` + ngữ cảnh và trả về *claim* `{role, confidence, evidence}` | Chain of Responsibility + Strategy | Xem bên dưới |
| 4. Assemblers | Gom các block đã gán vai trò thành từng khối miền | Builder | `FrontMatterAssembler`, `AbstractAssembler`, `BodyAssembler` (dựng cây section), `TableAssembler`, `FigureAssembler`, `EquationAssembler`, `ReferenceAssembler`, `BackmatterAssembler`, `FootnoteAssembler` |

| Tầng | Việc | Pattern | Ghi chú |
|---|---|---|---|
| 5. Normalizers | Hàm thuần `Manuscript → Manuscript`, chạy theo thứ tự | Pipeline | `UnicodeNFC` (bắt buộc cho tiếng Việt: NFD và NFC dễ lẫn), `VietnameseSpacing` (cái đang là `normalizeVietnameseWordSpacing`), `EndNoteCleanup`, `Punctuation`, `CitationLinker`, `XRefLinker`, `CaptionNumberStripper` |
| 6. Validate | Kiểm bằng zod, sinh `IngestReport`: số block không nhận diện được, claim độ tin cậy thấp, cảnh báo | Specification | Report đi vào ngăn kéo nội dung bóc tách |

## 4.1 Recognizer theo thứ tự ưu tiên

| Ưu tiên | Recognizer | Cách làm | Độ tin cậy |
|---|---|---|---|
| 1 | `StyleMapRecognizer` | Dùng `JournalProfile.styleMap` (ví dụ `ArticleTitle→title`, `Authors→authors`, `Abstract→abstract`, `Heading1→h1`, `SourceLine→source`, `Reference→reference`, `Guide*` / `Instruction` →**bỏ qua**) | 1.0 |
| 2 | `StructureRecognizer` | Theo vị trí và mẫu: caption `Table N.` / `Bảng N.` / `Figure N.` / `Hình N.`, `Keywords:` / `Từ khóa:`, dòng `[n]` trong danh mục, tiêu đề `References` / `Tài liệu tham khảo` | 0.8 |
| 3 | `FormatHeuristicRecognizer` | Cỡ chữ, in đậm, căn giữa, đoạn đầu tài liệu → title/authors | 0.5–0.6 |
| 4 | `AiRecognizer` (tùy chọn) | Chỉ gọi cho phần front-matter và các block còn mơ hồ (độ tin cậy < ngưỡng); trả về *đề xuất* chứ không ghi đè | 0.7 + cần người duyệt |

Khi hai claim xung đột, thắng bên có độ tin cậy cao hơn; mọi quyết định ghi vào `provenance` (nguồn, độ tin cậy, vị trí đoạn trong file gốc). Nhờ đó UI có thể tô cảnh báo vàng ở đúng block, và đội ingest biết chính xác recognizer nào sai.

```
interface Recognizer {
  id: string; priority: number;
  recognize(block: RawBlock, ctx: IngestContext): Claim | null;
}
// Thêm journal hay quy ước mới = thêm 1 recognizer hoặc 1 styleMap, không
đụng code cũ (Open/Closed).
```

### 4.2 Những điểm kỹ thuật cần giải quyết trong ingest

- **Công thức:** OMML → MathML (dùng XSLT chuẩn của Microsoft hoặc thư viện tương đương) → LaTeX; lưu cả `omml` gốc. Công thức không chuyển được thì `status: 'fallback-image'` (ảnh vector) và QC cảnh báo. Với MathType (OLE/WMF) coi là ảnh dự phòng cho tới khi có bộ chuyển.
- **Ảnh:** đọc tách riêng thành asset; nhận diện đúng định dạng bằng *magic bytes* chứ không dựa đuôi; PNG/JPEG/BMP giữ nguyên; TIFF → PNG; EMF/WMF/EPS/PDF → SVG/PDF vector (chuyển đổi phía server khi cần). Tính DPI và kích thước để QC.
- **Bảng:** dựng lưới trực tiếp từ `gridSpan` / `vMerge`, không đoán header từ chuỗi. Caption lấy từ style `TableCaption` (phía trên), `TableNote` và `SourceLine` (phía dưới).
- **Citation:** nếu có field Zotero/EndNote thì lấy dữ liệu có cấu trúc làm `Reference.csl`; nếu không, parse `[1]`, `[2–4]`, `[5,6]` thành node `cite` có `refIds`.
- **Chạy trong Web Worker** (Comlink) để không treo UI, và **viết isomorphic** để cùng package này chạy trong Node cho *batch processing* (xem mục 10).
- **`.doc` cũ:** gửi lên `DocConverterPort` (LibreOffice phía server); nếu chưa có BE thì chặn từ UI với thông báo hướng dẫn lưu thành `.docx`.

---

## 5. Journal Profile, dàn trang và typography

### 5.1 Profile là dữ liệu, không phải code

Hiện `JOURNAL_TEMPLATES` trong `templates.ts` chỉ có tên và màu, còn quy tắc khác nằm rải trong các Tab và `latex-template.ts`. Đề xuất mỗi tạp chí là một file JSON có schema (zod), có `version`, nạp qua `JournalProfileProvider`:

```
interface JournalProfile {
  id: 'VNUHCM-HS' | string; version: string;
  names: { vi: string; en: string; short: string }; color: string;
  languages: ('vi' | 'en')[];
  articleTypes: { id: string; label: Record<string, string>;
requiredSections: string[] }[];
  frontMatter: {
    abstract: { minWords: number; maxWords: number; structured: boolean };
    keywords: { min: number; max: number; separator: ';' | ',' };
    dateFormat: 'dd/MM/yyyy';
    doiPattern: string;                  // để QC đối chiếu DOI với
volume/issue
  };
  citation: { style: 'vancouver'; inText: 'bracket' | 'superscript';
rangeDash: '–' };
  styleMap: Record<string, string>;     // style Word -> vai trò ngữ nghĩa
(dùng ở ingest)
  layout: LayoutDefaults;               // khổ giấy, cột, lề, header/footer,
số trang
  typography: TypographyDefaults;       // font, cỡ chữ, leading, tracking,
giãn đoạn, hyphenation
  latex: { cls: string; engine: 'xelatex' | 'lualatex'; fonts: FontStack };
  qc: { ruleset: string[]; weights: Record<string, number>; overrides?:
object };
}
```

Các mâu thuẫn ở mục 1 (tóm tắt 150–250 hay 275–500 từ, từ khóa phân cách, định dạng ngày, 4 hay 8 loại bài, `[1]` hay chỉ số trên) đều được giải quyết bằng cách đặt vào profile. Khi tạp chí đổi quy định, chỉ sửa một file.

## 5.2 Hợp nhất cấu hình lúc render

```
resolved = deepMerge(globalDefaults, profile.layout/typography,
manuscript.layoutOverrides)
```

`LayoutSettingsDrawer` chỉ ghi vào `layoutOverrides` những gì khác profile, nên đổi tạp chí đích không làm mất tùy chỉnh hợp lệ và có thể bấm *Đặt lại về mặc định của tạp chí*.

## 5.3 Typography (đáp ứng nhóm yêu cầu Typography)

| Yêu cầu | Nơi cấu hình | Cách thực thi trong LaTeX |
|---|---|---|
| Font, cỡ chữ, leading | `typography.fonts`, `size`, `leading` | `fontspec`, `\linespread`; có **fallback font thật** (danh sách ưu tiên), không còn thay chuỗi MinionPro→Times New Roman bằng string replace |
| Tracking | `typography.tracking` | `microtype` (`letterspace`) |
| Giãn đoạn | `paragraphSpacing`, `indent` | `\parskip`, `\parindent` |
| Hyphenation | `typography.hyphenation { lang, leftMin, rightMin, enabled }` | `polyglossia`/`babel` theo `Manuscript.lang`; từng đoạn có thể mang `lang` riêng (tóm tắt tiếng Anh trong bài tiếng Việt). Tiếng Việt chủ yếu âm tiết đơn nên mặc định tắt; tiếng Anh bật |
| Widow/orphan | `typography.widowOrphan` | `\clubpenalty`, `\widowpenalty` |
| Lề, 1/2 cột, A4 | `layout.page` | `geometry`, `multicol`/`twocolumn` |
| Header/footer, running title, số trang | `layout.header/footer/pageNumbering` | `fancyhdr` (chuyển logic hiện có `pageNumbering` sang đây) |

## 5.4 Trang đầu và lớp `.cls` do bộ phận dàn trang sở hữu

PDF mẫu ID630_HS cho thấy trang 1 là một bố cục đặc biệt: cột trái chứa Liên hệ/Lịch sử/DOI/Bản quyền/QR, khung Tóm tắt nền xanh nhạt, hộp *Trích dẫn bài báo này ở* chân trang, đầu trang lặp tên tạp chí và số trang. Những thứ này **không nên sinh bằng TypeScript**. Đề xuất:

- Mỗi tạp chí có một lớp `structpaper-hs.cls` (hoặc `.sty`) do người làm TeX sở hữu, định nghĩa các macro như `\spTitle`, `\spAuthors`, `\spAbstract`, `\spSidebar`.
- LaTeX emitter chỉ sinh *lời gọi macro* với nội dung đã escape. Đổi thiết kế trang = sửa `.cls`, không đụng TypeScript.
- Số thứ tự (Figure, Table, Equation) do `computeNumbering` cấp, không dựa vào bộ đếm LaTeX. Hai bên khớp nhau khi preview và khi xuất.

---

# 6. Kiến trúc Editor

## 6.1 Một store, một nguồn sự thật

Bỏ mô hình hai tầng `formData` ↔ `PaperContext` và `isDirty` set tay. Thay bằng **một store** (Zustand + Immer hoặc Redux Toolkit) giữ `Manuscript`. Hiệu năng gõ phím đạt được bằng selector theo ID và `React.memo` ở từng block, không phải bằng bản nháp cục bộ:

- Component đoạn văn chỉ đăng ký `selectBlock(id)`, nên gõ một đoạn không render lại cả bài.
- Ô nhập giữ state cục bộ và commit vào store sau khoảng 150 ms (debounce).
- Danh sách block dùng ảo hóa (`react-virtuoso`) vì bài tổng quan dài.
- `isDirty` là giá trị suy ra: `rev` hiện tại khác `rev` đã lưu.

## 6.2 Command + History (Command, Memento)

Mọi thay đổi đi qua `CommandBus`; không component nào gọi `setState` lên tài liệu trực tiếp.

```
interface Command<P = unknown> { type: string; payload: P }
// handler: (doc, payload) => { doc: Manuscript; patches: Patch[]; inverse:
Patch[] }

bus.dispatch({ type: 'block/insert', payload: { after: blockId, block } });
bus.dispatch({ type: 'reference/remove', payload: { id } });  // numbering tự
cập nhật vì là dữ liệu dẫn xuất
bus.dispatch({ type: 'table/mergeCells', payload: { id, range } });
```

Lợi ích: **undo/redo** miễn phí (dùng patch ngược của Immer), **autosave** chỉ cần lắng nghe luồng patch, **op-log** đẩy lên BE về sau, **lịch sử revision** là chuỗi patch, và mỗi command test được độc lập không cần React. Sửa tay `isDirty`, `setPaper`, `updatePaper` đều biến mất.

## 6.3 Feature module (cắt dọc) và Block registry

Mỗi tab trong `EditorTabNav` là một feature package, có cùng khung:

```
feature-references/
  commands/        # reference/add, remove, reorder, parseRaw
  selectors/       # danh sách có số thứ tự, trạng thái cite
  ui/              # ReferencesTab, ReferenceItem
  qc/              # luật QC riêng của tính năng
  index.ts         # export { tab, commands, qcRules }
```

Với nội dung thân bài, dùng **Block registry** để một loại block là một đơn vị độc lập xuyên các tầng:

```
registerBlock('table', {
  schema: TableBlockSchema,
  Editor: TableEditor,            // feature-body/table
  toLatex: tableToLatex,          // exporters/latex/blocks/table.ts
  toDocx: tableToDocx,
  toJats: tableToJats,
  qc: tableRules,
});
```

TypeScript buộc kiểu `Record<BlockType, Handler>` ở mỗi tầng: thêm block mới (ví dụ `equation`) mà quên viết exporter sẽ **lỗi biên dịch**, nên không còn trường hợp sót. Ba người làm ba block khác nhau không đụng nhau.

## 6.4 Soạn thảo văn bản có cấu trúc

Ô `textarea` hiện tại không thể giữ in nghiêng *H. pylori*, chỉ số dưới $FEV_1$, citation và cross-ref như những đối tượng thật. Đề xuất dùng **TipTap/ProseMirror** cho đoạn văn và tiêu đề:

- Schema ProseMirror ánh xạ 1-1 với `Inline[]`; `cite` và `xref` là *node* có thể bấm, không phải regex tô màu.
- Gõ `[` mở bộ chọn tài liệu tham khảo; chèn *Figure/Table/Equation* bằng danh sách mục tiêu.
- Dán từ Word/Excel đi qua cùng bộ chuẩn hóa của ingest.
- Công thức: preview bằng KaTeX, sửa bằng MathLive; bảng giữ modal dạng bảng tính hiện có, đọc/ghi lưới mới.
- Chi phí: đây là phần tốn công nhất của Editor, nên đặt ở giai đoạn 4 (mục 14). Có thể làm sau khi store và command đã ổn.

### 6.5 Preview hai tầng

Hiện chỉ có PDF bằng XeLaTeX nên mỗi lần đổi là chờ 1–2 giây và `iframe` nhấp nháy. Đề xuất:

| Tầng | Nội dung | Khi nào |
|---|---|---|
| Preview nhanh | React render từ AST theo profile, gần giống dàn trang | Tức thì, trong lúc gõ |
| Preview chính xác | PDF từ compile service, hiển thị bằng `pdf.js` (giữ nguyên vị trí cuộn, đánh dấu lỗi trên trang) | Debounce 1–2 s sau khi ngừng gõ hoặc bấm biên dịch |

Từ khi emitter chèn comment `%%BLOCK:<id>` vào TeX, có thể ánh xạ ngược **lỗi LaTeX → block** (bấm lỗi để nhảy tới đúng đoạn) và dùng SyncTeX cho **bấm vào PDF → nhảy tới block**.

### 6.6 Bus sự kiện cho các việc nền

Các việc nền đăng ký lắng nghe luồng patch thay vì được gọi tay trong `UnifiedEditorView`:

```
document patch ─┬─▶ autosave (debounce 1 s)  ─▶ Repository.save
                ├─▶ QC engine (worker, debounce, chỉ tính lại block đổi) ─▶ qcStore
                ├─▶ compile scheduler (debounce, hủy job cũ) ─▶ previewStore
                └─▶ outbox (khi có BE) ─▶ đồng bộ
```

`UnifiedEditorView` còn lại chỉ là khung bố cục: chia đôi màn hình, dock, tab. Không còn logic nghiệp vụ.

---

## 7. Quality Control / Preflight (rule engine)

Hiện `computePreflightCheck` chấm 100 điểm, và `docx-extractor.ts` vừa trích xuất vừa chấm. Yêu cầu phần mềm đòi preflight rộng hơn nhiều (font missing, image resolution, overset text, lỗi link, lỗi numbering). Cách làm bền vững là **mỗi luật là một đơn vị nhỏ** (Specification pattern):

```
interface Rule {
  id: string;                          // 'ref.uncited', 'fig.dpi',
'meta.doi-volume'
  severity: 'error' | 'warning' | 'info';
  stage: 'static' | 'post-compile';    // post-compile đọc log/PDF
  appliesTo(profile: JournalProfile): boolean;
  run(ctx: QcContext): Issue[];        // ctx có doc, numbering, profile,
compileReport?
}
interface Issue { ruleId; severity; message; target: { blockId?: Id; path?:
string }; fix?: Command }
```

- Profile chọn bộ luật qua `qc.ruleset` và trọng số điểm; tạp chí khác nhau có luật khác nhau mà không rẽ nhánh trong code.
- `fix` là một Command: nút *Sửa nhanh* trong `PreflightQCTab` chỉ việc dispatch.
- Chạy trong worker, debounce, memoize theo block để không tính lại cả bài.

| Nhóm | Ví dụ luật | Nguồn dữ liệu |
|---|---|---|
| Metadata | Thiếu tiêu đề/tác giả/email liên hệ; số từ khóa; độ dài tóm tắt theo profile; **DOI không khớp volume/issue** (lỗi có thật trong PDF mẫu) | doc + profile |
| Cấu trúc | IMRAD theo `articleType`; mục bắt buộc thiếu (Ethics, Funding, Conflict, Data availability); style `Guide*` còn sót trong nội dung | doc |
| Numbering | Figure/Table nhắc sau khi đã xuất hiện, hoặc không được nhắc; xref trỏ vào block không tồn tại; thứ tự đánh số khác thứ tự nhắc lần đầu | `computeNumbering` |
| Reference | Cite không có reference; reference không được cite; trùng DOI; reference parse `failed`; thiếu năm/tập/trang | doc |

| Nhóm | Ví dụ luật | Nguồn dữ liệu |
|---|---|---|
| Hình | DPI thấp, kích thước quá nhỏ cho khổ cột, thiếu caption, thiếu dòng *Nguồn* | `AssetMeta` |
| Bảng | Bảng là ảnh; thiếu caption hoặc dòng nguồn; bảng vượt quá chiều rộng | doc |
| Công thức | Chuyển đổi thất bại ( `fallback-image` ); công thức thiếu số hoặc thiếu tham chiếu | doc |
| Sau biên dịch | `Overfull \hbox` (overset text), font bị thay thế, `Reference undefined` , hình thiếu, font chưa nhúng, ảnh dưới DPI | log TeX, `pdffonts` , `pdfimages` do compile worker trả về |

Báo cáo ingest (mục 4) và QC dùng chung kiểu `Issue` , nên giao diện chỉ có một danh sách vấn đề duy nhất.

---

# 8. Exporters và Compile service

## 8.1 Một AST, nhiều đầu ra (Strategy + Visitor)

Tất cả đầu ra đọc cùng `Manuscript` đã qua `computeNumbering` và `resolvedProfile` :

| Exporter | Đầu ra | Ghi chú |
|---|---|---|
| `LatexExporter` | `article.tex` + `references.bib` | Hợp nhất `exportToLatex` và `generateFullJournalLatex` . Dùng *LaTeX writer* nhỏ với hàm `escapeTex` bắt buộc cho mọi text, có golden test |
| `JatsExporter` | JATS XML 1.3 | Đáp ứng yêu cầu Online publishing. Có sẵn nhờ cây section, `xref` , `cite` , `Reference.csl` và `lang` |
| `DocxExporter` | `.docx` | Dùng lại bộ style của Template_EN/VI nên Word xuất ra khớp với đầu vào |

| Exporter | Đầu ra | Ghi chú |
|---|---|---|
| `HtmlExporter` | HTML | Dùng chung với preview nhanh |
| `ZipPackager` | `tex + bib + images + (xml)` | Gói nộp tòa soạn hoặc Overleaf |

Mỗi exporter là một thư mục với mỗi block một file ( `blocks/table.ts` , `blocks/equation.ts` …), thỏa kiểu `Record<BlockType, Handler>` (mục 6.3).

## 8.2 Compile service tách riêng

Loại bỏ việc biên dịch ở hai nơi. Next.js không còn route compile trong môi trường thật; `compiler-worker` là một container riêng, Spring Boot điều phối.

```
Editor ──(manifest: tex + danh sách assetId)──▶ Spring /api/v1/compile
          ◀──────────── jobId, SSE trạng thái ────────────┐
Spring ─ kiểm tra cache theo hash(tex + assets + profile) |
       ─ asset thiếu thì client mới upload (HEAD theo sha256)
       ─ đẩy job vào hàng đợi ──▶ compiler-worker ──▶ PDF + log + báo cáo preflight
```

- **Không gửi lại base64 mỗi lần.** Ảnh upload một lần theo hash; manifest chỉ liệt kê hash. Bỏ luôn hack alias `fig1.png/fig1.jpg` .
- **Cache** theo hash nội dung (client giữ thêm một cache nhỏ ở IndexedDB), **hủy job cũ** khi có job mới của cùng bản thảo, **debounce** ở client.
- **Sandbox bắt buộc** vì TeX do hệ thống sinh từ nội dung người dùng: chạy `--no-shell-escape` , `openin_any=p` , `openout_any=p` , container không có mạng, giới hạn CPU/RAM/thời gian, thư mục tạm riêng mỗi job, người dùng không root. Vẫn escape mọi text và chặn `\input` , `\write18` do người dùng gõ vào (kể cả khi dán).
- **Cổng `CompilerPort` :** dev dùng `LocalCliCompiler` (xelatex/tectonic), production dùng `RemoteCompiler` . UI không biết khác biệt.
- **Dây chuyền prepress cho PDF xuất bản:** `xelatex → Ghostscript (PDF/X hoặc PDF/A) → kiểm font nhúng (pdffonts) → kiểm DPI (pdfimages) → nhúng metadata (hyperxmp) → bookmarks` . Kết quả preflight quay về QC (stage `post-compile` ).
- **Rủi ro cần quyết sớm:** PDF/X và PDF gắn thẻ truy cập (accessibility tagging) hỗ trợ tốt hơn trên LuaLaTeX so với XeLaTeX. Nếu truy cập là bắt buộc, nên làm thử LuaLaTeX trước khi dồn công vào `.cls` . Font thương mại (MinionPro, MyriadPro) cần

giấy phép cho worker; nếu chưa có, profile phải khai báo font thay thế và QC hiện cảnh báo *đang dùng font thay thế*.

---

# 9. Persistence offline-first với IndexedDB (giảm tải BE), sẵn sàng cho Spring Boot + MySQL

## 9.1 Quyết định

- **IndexedDB là nơi làm việc chính** của trình duyệt: mở bài, sửa, undo, preview đều không cần BE. Dùng **Dexie** (API gọn, migration theo version, hỗ trợ transaction và index).
- Mọi truy cập đi qua `ManuscriptRepository` và `AssetRepository` (mục 2.2). Sau này BE chỉ là một adapter bổ sung và một *decorator* đồng bộ, **không sửa Editor**.
- Bỏ `sessionStorage`: giới hạn khoảng 5 MB, mất khi đóng tab, và `try {} catch {}` hiện tại nuốt lỗi quota một cách im lặng.

## 9.2 Lược đồ Dexie

| Bảng | Khóa / index | Nội dung | Ghi chú |
|---|---|---|---|
| `manuscripts` | `id`, index `[journalId+updatedAt]`, `status`, `syncState` | `Manuscript` JSON **không chứa blob**; `rev`, `schemaVersion`, `title`, `updatedAt` | Danh sách *Bản thảo của tôi* chỉ cần cột nhẹ (`title`, `updatedAt`, `status`) để khỏi nạp cả bài |
| `revisions` | `id`, index `[manuscriptId+rev]` | Chuỗi patch giữa các snapshot; snapshot đầy đủ mỗi N revision hoặc khi người dùng bấm *Lưu mốc* | Đáp ứng yêu cầu Revision và proofing; có thể khôi phục |
| `assets` | `hash` (sha256) | `Blob`, `mime`, `size`, `width`, `height`, `dpi`, `vector` | Một ảnh dùng ở nhiều bản thảo chỉ lưu một lần |

| Bảng | Khóa / index | Nội dung | Ghi chú |
|---|---|---|---|
| `assetRefs` | `[manuscriptId+hash]` | Quan hệ bản thảo ↔ asset | Dọn rác (GC) asset mồ côi |
| `ingestSources` | `manuscriptId` | Blob `.docx` gốc + `IngestReport` | Dùng cho *Nội dung bóc tách*, nhập lại khi có bản sửa (Revision) |
| `outbox` | `++seq`, `manuscriptId` | Thao tác chờ đẩy lên BE (`{op, rev, patches}`), `attempts` | Chỉ dùng khi bật đồng bộ |
| `compileCache` | `hash` | PDF Blob + thời gian | Giới hạn LRU theo dung lượng |
| `settings` | `key` | Profile đã chọn, cấu hình UI, `consentAi` | Không lưu API key |

## 9.3 Bảo đảm độ bền

- **Autosave** debounce khoảng 1 giây từ luồng patch; ghi `manuscripts` và `revisions` trong **cùng một transaction** để không lệch.
- Gọi `navigator.storage.persist()` và theo dõi `navigator.storage.estimate()`. Khi gần hết quota, hiển thị cảnh báo và đề nghị xuất ZIP; bắt `QuotaExceededError` thật, không nuốt lỗi.
- **Safari/ITP** có thể xóa dữ liệu website sau thời gian không dùng, nên IndexedDB không được coi là kho lưu trữ duy nhất khi bài quan trọng. Đây chính là lý do cần BE đồng bộ, và trước đó cần nút *Xuất bản sao lưu* (`.structpaper` = zip gồm JSON và asset).
- **Nhiều tab:** dùng `Web Locks API` hoặc `BroadcastChannel` để một bản thảo chỉ được sửa ở một tab; tab khác mở chế độ chỉ đọc.
- **Migration:** mỗi lần đổi lược đồ thì tăng `db.version(n)` và có hàm `upgrade`; riêng `Manuscript` có `schemaVersion` và chuỗi `migrate(vN→vN+1)` có test với fixture.

### 9.4 Chiến lược đồng bộ khi có BE

`SyncedRepository` là decorator quanh `IndexedDbRepository`: ghi cục bộ trước, đưa vào `outbox`, đẩy khi có mạng.

| Giai đoạn | Cơ chế | Khi nào dùng |
|---|---|---|
| S1 | Đồng bộ theo **bản sao**: mỗi bản thảo có `rev`; lưu lên BE bằng `If-Match: <rev>` (khóa lạc quan). Xung đột thì giữ cả hai (tạo *bản xung đột*) và để người dùng chọn | Tác giả làm một mình hoặc luân phiên; đủ cho giai đoạn đầu |
| S2 | Đồng bộ theo **chuỗi patch** (op-log) đã có sẵn từ Command | Khi cần giảm băng thông và lưu lịch sử chi tiết trên BE |
| S3 | CRDT (Yjs gắn với ProseMirror) | Chỉ khi thật sự cần nhiều người sửa đồng thời |

Khuyến nghị bắt đầu từ S1. Vì mọi thay đổi đã là Command nên nâng lên S2 không phải viết lại Editor.

---

## 10. Backend Spring Boot + MySQL

### 10.1 Cấu trúc (modular monolith, tách service khi thật cần)

```
vn.edu.vnu.structpaper
  identity/     # đăng nhập (JWT/OIDC), vai trò: Author, Editor, Production,
Admin
  manuscript/   # bản thảo, revision, quyền truy cập
  asset/        # upload theo sha256, metadata ảnh, lưu S3/MinIO hoặc
filesystem
  journal/      # JournalProfile (version), template .cls
  compile/      # job, hàng đợi, kết quả, cache
  ingest/       # chuyển .doc → .docx (LibreOffice), batch ingest bằng Node
worker
  ai/           # proxy OpenAI, ghi log sử dụng
  common/       # lõi chuẩn, audit, cấu hình
```

Mỗi gói có controller, service, repository, DTO riêng; không gọi chéo vào repository của gói khác (qua service/interface). Kiểm bằng ArchUnit.

## 10.2 Lược đồ MySQL (rút gọn)

| Bảng | Cột chính | Ghi chú |
|---|---|---|
| `users` | id, email, name, role | |
| `journals` / `journal_profiles` | journal_id, version, profile_json (JSON), active | Profile có version; bản thảo ghi nhớ version đã dùng |
| `manuscripts` | id (ULID), owner_id, journal_id, title, doi, status, rev, schema_version, **doc_json (JSON)**, updated_at | `@Version` cho khóa lạc quan; cột phụ (title, doi, status) để tìm kiếm mà không cần mở JSON |
| `manuscript_revisions` | manuscript_id, rev, kind (patch/snapshot), payload, author_id, created_at | Khớp `revisions` ở IndexedDB |
| `assets` | sha256 (PK), size, mime, width, height, dpi, storage_key | |
| `manuscript_assets` | manuscript_id, sha256 | |
| `compile_jobs` | id, manuscript_id, rev, tex_hash, status, started_at, finished_at, pdf_key, log_key, report_json | |
| `batches` | id, owner_id, status, total, done | Batch processing |
| `audit_log` | actor, action, target, at | |

`Manuscript` là một aggregate nên lưu dạng JSON trong một dòng (MySQL 8 JSON), đồng thời phi chuẩn hóa vài cột để truy vấn. Dùng **Flyway** cho migration.

## 10.3 Hợp đồng FE–BE

- **OpenAPI** là nguồn sự thật. Sinh client TypeScript ( `openapi-typescript` hoặc Orval) và DTO Java từ cùng một file, để hai bên không lệch.

- `Manuscript` được kiểm băng **JSON Schema sinh từ zod**; BE (thư viện json-schema-validator) từ chối tài liệu sai `schemaVersion`. Contract test chạy trong CI cả hai phía.
- Endpoint chính:

| Endpoint | Việc |
|---|---|
| `GET/PUT /api/v1/manuscripts/{id}` | Đọc và ghi bản thảo ( `If-Match` ) |
| `GET /api/v1/manuscripts/{id}/revisions` | Lịch sử |
| `HEAD/PUT /api/v1/assets/{sha256}` | Kiểm tra và upload ảnh theo hash |
| `POST /api/v1/compile` rồi `GET /api/v1/compile/{jobId}` (hoặc SSE) | Biên dịch |
| `GET /api/v1/journals/{id}/profile` | Profile có version |
| `POST /api/v1/ingest/convert` | `.doc` → `.docx` |
| `POST /api/v1/batches` | Nạp zip nhiều `.docx`, xử lý hàng loạt |
| `POST /api/v1/ai/extract` | Proxy AI |

### 10.4 Batch/API/workflow (yêu cầu Automation)

Nhờ `ingest-docx` isomorphic, chạy cùng package trong một **Node worker** nối vào hàng đợi: `POST /batches` nhận zip, mỗi `.docx` thành một job (ingest → QC → compile), trả báo cáo tổng hợp. Không viết parser lần thứ hai bằng Java.

---

## 11. AI: trợ lý đề xuất, không ghi đè

- **Khóa API chỉ ở BE.** Bỏ `NEXT_PUBLIC_OPENAI_API_KEY` và ô nhập `sk-…` ở client. Client gọi `AiPort` → `POST /api/v1/ai/extract`.
- **Chỉ gửi phần cần thiết**: front-matter và các block mà recognizer đánh giá mơ hồ, không gửi cả bản thảo. Giảm token, giảm rủi ro lộ bản thảo chưa công bố. Có công tắc *đồng ý gửi nội dung ra dịch vụ AI* ( `consentAi` ) và ghi log sử dụng.
- **Kết quả là đề xuất dạng Command** kèm độ tin cậy; giao diện hiển thị khác biệt để người dùng chấp nhận hoặc từ chối từng thay đổi. Không còn "merge đè lên dữ liệu OpenXML".

- Dùng đầu ra có cấu trúc (JSON Schema) và prompt có version trong `ai-client`; test bằng bộ fixture, mô hình giả (`MockAi`) cho CI.
- Tên mô hình nằm trong cấu hình BE; UI không hard-code danh sách model.

---

## 12. Bảng design pattern (dùng ở đâu, để làm gì)

| Pattern | Áp dụng | Giải quyết vấn đề |
|---|---|---|
| Hexagonal (Ports & Adapters) | Repository, Compiler, AI, Converter, Profile | Đổi IndexedDB ↔ Spring, local ↔ remote mà không sửa Editor |
| Anti-corruption layer | `ingest-docx`: kiểu OOXML không lọt ra ngoài package | Giữ lõi miền sạch |
| Pipeline / Chain of Responsibility | Các tầng ingest, bộ Normalizer | Mỗi tầng một người làm, thêm bước không sửa bước cũ |
| Strategy | Recognizer, Exporter, Compiler, chính sách Track Changes | Thay thuật toán theo ngữ cảnh |
| Visitor | Duyệt AST: numbering, export, QC | Một cách duyệt, nhiều xử lý |
| Registry / Plugin | Block registry, Journal Profile | Thêm block hoặc tạp chí không chạm code cũ (Open/Closed) |
| Command + Memento | Mọi thao tác Editor, undo/redo, revision | Nhật ký thay đổi duy nhất |
| Repository + Decorator | `SyncedRepository` bọc `IndexedDbRepository` | Thêm đồng bộ mà không đổi interface |
| Observer / Event bus | Autosave, QC, compile, outbox | Bỏ logic điều phối khỏi `UnifiedEditorView` |
| Specification | Luật QC | Luật nhỏ, ghép được, bật tắt theo profile |
| Builder | Assembler (ingest), LaTeX writer | Dựng cấu trúc phức tạp từng bước |

| Pattern | Áp dụng | Giải quyết vấn đề |
|---|---|---|
| Factory | `createEmptyManuscript(profile)`, tạo block | Thay `createEmptyPaper` và các hàm rải rác |
| Adapter | OpenAI, TeX engine, LibreOffice | Che chi tiết bên ngoài |
| Derived data (selector) | `computeNumbering` | Số thứ tự không bao giờ lệch |

Không dùng hết cùng lúc ở ngày đầu. Ba pattern **bắt buộc có sớm** là Command, Repository và Registry (cùng Derived data); phần còn lại thêm dần theo lộ trình.

---

## 13. Chiến lược kiểm thử và chất lượng mã

| Loại | Cách làm | Ví dụ |
|---|---|---|
| Golden ingest | `fixtures/*.docx` → so sánh JSON kỳ vọng (snapshot) | Template_EN, Template_VI, 8 bài mẫu hiện có; mỗi lỗi parser sau này thêm một fixture |
| Unit theo tầng | Đoạn XML nhỏ cho từng assembler/normalizer | `VietnameseSpacing`: `Nguy ễ n` → `Nguyễn`; NFD→NFC |
| Property-based | `fast-check` cho `computeNumbering` | Chèn/xóa ngẫu nhiên block rồi kiểm số liên tục, không trùng, cite khớp |
| Golden export | LaTeX/JATS/DOCX so với file mẫu | Có `escapeTex` cho ký tự đặc biệt, tiếng Việt |
| Visual regression | Biên dịch bài mẫu, `pdftoppm` + `pixelmatch` với **PDF tham chiếu** (ID630_HS) | Bắt lệch layout khi sửa `.cls` |
| Contract | OpenAPI + JSON Schema chạy ở cả FE và BE | Sai `schemaVersion` bị từ chối |
| E2E | Playwright: upload → sửa → autosave → F5 → khôi phục → xuất PDF | Kiểm đúng lỗi mất dữ liệu hiện nay |
| Kiến trúc | `dependency-cruiser`, ArchUnit | Vi phạm tầng thì fail CI |

Quy ước chung: TypeScript `strict`, zod ở mọi biên (nhập file, đọc IndexedDB, gọi API), lỗi trả về dạng `Result`/`Issue` thay vì `catch {}` im lặng, mỗi package có README ghi trách nhiệm và API công khai.

---

## 14. Lộ trình chuyển đổi (strangler, không viết lại từ đầu)

Mục tiêu: sản phẩm luôn chạy được ở cuối mỗi giai đoạn.

| Giai đoạn | Việc chính | Kết quả kiểm chứng |
|---|---|---|
| 0. Đóng băng hành vi | Dựng monorepo, CI, `dependency-cruiser`. Viết *characterization test* cho parser và exporter hiện tại bằng 8 bài mẫu và 2 template | Biết chắc đang chạy thế nào trước khi đổi |
| 1. Lõi `doc-model` | Manuscript v2, zod, `computeNumbering`, `migrateV1toV2` + adapter ngược cho UI cũ | Số thứ tự và cite/xref tính đúng; property test xanh |
| 2. Lưu trữ | `storage` với Dexie (manuscripts, assets, revisions), autosave, trang *Bản thảo của tôi*, nút sao lưu `.structpaper`. Tách base64 thành asset | F5/đóng tab không mất dữ liệu; bài nhiều ảnh lớn vẫn lưu được |
| 3. Ingest pipeline | Chạy song song parser cũ và mới, so kết quả. Làm `StyleMapRecognizer` trước (ăn ngay Template_EN/VI), sau đó assemblers, OMML→MathML→LaTeX, ảnh/magic bytes. Chuyển vào Web Worker | Golden test xanh; parser cũ được gỡ khi mới đạt ngang |
| 4. Editor core | Store + CommandBus + history; chuyển từng tab sang feature module; Block registry; thay textarea bằng ProseMirror cho đoạn văn | Undo/redo; `UnifiedEditorView` không còn logic nghiệp vụ |
| 5. QC + Exporters | Rule engine, gộp hai bộ sinh LaTeX thành một, thêm exporter JATS, tách `.cls` cho HS | Báo cáo QC thống nhất; XML JATS hợp lệ |

| Giai đoạn | Việc chính | Kết quả kiểm chứng |
|---|---|---|
| 6. Backend | Spring Boot (manuscript, asset, journal, identity), `SyncedRepository` S1, compiler-worker có sandbox và cache, AI proxy | Mở bài ở máy khác; compile an toàn |
| 7. Production | PDF/X + prepress, post-compile preflight, proofing (comment trên PDF), batch, S2 (đồng bộ theo patch) | Đáp ứng đầy đủ file yêu cầu |

Giai đoạn 3 và 4 có thể làm song song nhờ `doc-model` đã chốt ở giai đoạn 1.

---

## 15. Chia việc theo team (hết cảnh "dồn cục")

| Nhóm | Sở hữu (CODEOWNERS) | Phụ thuộc |
|---|---|---|
| Model & Core | `doc-model`, `journal-profiles` (schema) | không |
| Ingest | `ingest-docx` (mỗi assembler/recognizer một người, ví dụ Bảng, Hình, Công thức, Reference) | `doc-model` |
| Editor | `editor-core`, `feature-*` | `doc-model`, `storage`, `qc-engine` |
| Xuất bản | `exporters`, `compiler-worker`, `.cls` cho từng tạp chí | `doc-model` |
| QC | `qc-engine` và luật | `doc-model` |
| Backend | `apps/api`, `contracts/` | OpenAPI |
| Hạ tầng | CI, Docker, observability | tất cả |

Như vậy việc "Bảng", "Hình", "Equation", "Reference", "Hyphenation" trong file yêu cầu là các thư mục và người sở hữu khác nhau: `ingest/assemblers/table.ts`, `exporters/latex/blocks/table.ts`, `feature-body/table/`, `qc-engine/rules/table.ts`, và typography nằm trong profile cùng `.cls`. Một thay đổi ở bảng không buộc ai đọc 2.100 dòng của parser.

# 16. Ma trận phủ yêu cầu phần mềm

| Yêu cầu (file yêu cầu) | Nơi đáp ứng |
|---|---|
| Input DOCX/DOC, Track Changes | `ingest-docx` (chính sách Track Changes, `.doc` qua `DocConverterPort`) |
| Nhận diện Heading, tác giả, tóm tắt, từ khóa, reference… | `StyleMapRecognizer` + `StructureRecognizer` + assemblers |
| Template riêng từng journal, nhiều loại bài | `JournalProfile` (`articleTypes`, `styleMap`, `layout`) |
| Layout 1/2 cột, A4, header/footer, số trang | `profile.layout` + `layoutOverrides` + `.cls` |
| Typography (font, leading, tracking, hyphenation) | `profile.typography` → `fontspec`, `microtype`, `polyglossia` |
| Hình: nhiều định dạng, vector, multi-panel, kiểm DPI | Asset store + `figure.panels` + QC `fig.*` |
| Bảng: nhiều dòng/cột, ngang, qua trang, lặp header | Lưới rowSpan/colSpan + `longtable` / `sidewaystable` + `Table N (continued)` |
| Công thức: OMML/MathType/LaTeX, đánh số, cross-ref | Block `equation` + OMML→MathML→LaTeX + `xref` |
| References Vancouver, renumber, link citation | `Reference.csl`, `cite` theo ID, `computeNumbering` |
| Cross-reference tự cập nhật | Node `xref` theo ID |
| DOI/ORCID/metadata | `FrontMatter` + QC + hyperxmp trong PDF |
| PDF publication/prepress | Compile worker + Ghostscript/PDF/X + preflight sau biên dịch |
| XML/JATS | `JatsExporter` |
| Proofreading PDF | Bản compile có version + comment (giai đoạn 7) |

| Yêu cầu (file yêu cầu) | Nơi đáp ứng |
|---|---|
| Revision không dàn lại từ đầu | `revisions` + `ingestSources` + nhập lại theo `provenance` |
| Batch/API/workflow | Node worker dùng lại `ingest-docx`, `POST /batches` |
| Quality control | `qc-engine` |

---

## 17. Các điểm cần bạn xác nhận

1. **Cộng tác thời gian thực** có trong phạm vi không? Nếu không, dừng ở đồng bộ S1/S2 và bỏ CRDT.
2. **PDF/X và PDF gắn thẻ truy cập có bắt buộc không?** Nếu có, nên thử LuaLaTeX sớm, trước khi viết `.cls`.
3. **Font thương mại (MinionPro, MyriadPro)** có giấy phép dùng trên máy chủ không? Nếu không, cần chọn font thay thế chính thức.
4. **Trích dẫn trong bài:** theo template là `[1]`, còn PDF mẫu là chỉ số trên. Mặc định cho từng tạp chí là gì?
5. **Mức tin cậy với template:** có bắt buộc tác giả dùng Template_EN/VI chuẩn (ingest theo style, gần như chính xác tuyệt đối) hay vẫn phải chấp nhận Word tự do (cần heuristic/AI nhiều hơn)?

Sau khi bạn chốt các điểm trên mình có thể viết tiếp đặc tả chi tiết cho `doc-model` (zod schema đầy đủ), `StyleMapRecognizer` cho Template_VI/EN, và lược đồ Dexie dưới dạng mã.